

## Deep Learning Models vs LLMs

### 1) The Big Idea

**Deep learning models** are *any* models built with neural networks (vision, audio, text, time-series, recommendation, generative images, etc.).

**LLMs (Large Language Models)** are a **specific type of deep learning model** built mainly for **language understanding + generation**, usually using the **Transformer** architecture.

Simple diagram:

#### Deep Learning (Neural Networks)

- Vision models (CNNs / Vision Transformers)
- Speech/audio models
- Recommenders and forecasting models
- Generative image models (diffusion, GANs)

#### Language Models

LLMs ( Large Transformer language models)

### Analogy

- Deep learning models = “all kinds of doctors” (cardiologist, dentist, surgeon, etc.)
- LLMs = “language specialist doctor” (great at reading/writing/talking—less suited for tasks that are purely visual or numeric unless paired with other tools)

### 2) What makes a model an “LLM”?

An LLM is typically:

- **Transformer-based** (attention + feed-forward blocks repeated many times)
- Trained on **massive text datasets** to learn patterns in language
- Used by generating text **token-by-token** (like very powerful autocomplete)

**Tiny Mental Model:**



### 3) Deep Learning Models That Are Not LLMs (and what they're for)

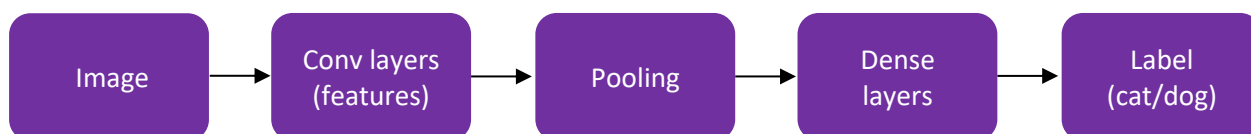
These are still deep learning, just not “language-first”:

#### A) Vision Model

Used for: Image classification, object detection, medical imaging

Think: “learn edges → shapes → objects.”

Diagram (Conceptual):



#### B) Diffusion / GAN (Image Generation)

Used for: generating images, enhancing resolution, style transfer  
(Generative, but not primarily language.)

#### C) Forecasting / Anomaly Detection Nets

Used for: sensor data, time-series, fraud signals, monitoring metrics.

**Key point:** LLMs are amazing at *language*, but for many “business ML” tasks (risk/churn/pricing), classic ML or smaller deep nets can still be the best choice.

### 4) LLMs vs Other Deep Learning Models: Quick Comparison

| Topic              | Other Deep Learning Models           | LLMs                                            |
|--------------------|--------------------------------------|-------------------------------------------------|
| Main data type     | Images/audio/time-series/tables      | Text (often plus multimodal)                    |
| Typical output     | Label, score, boxes, mask, signal    | Generated text / structured text                |
| Strength           | Pattern recognition in that modality | Language reasoning, summarising, drafting, chat |
| Weakness           | Needs task-specific design           | Can hallucinate; need grounding for facts       |
| Production pattern | Train → serve model                  | Often “LLM + retrieval + tools + guardrails”    |

## 5) How LLMs Are Used in Production Apps (Practical Patterns)

### Pattern 1: RAG (Retrieval-Augmented Generation)

Best for: “Answer using *our* documents”

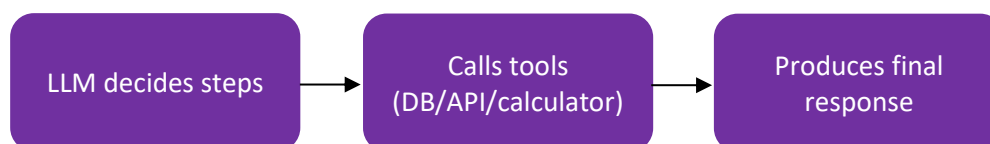
#### User Question

- Retrieve relevant docs (vector search)
- Send docs + question into LLM
- Answer with citations

Why: reduces hallucinations by grounding answers in retrieved text.

### Pattern 2: Tool Use

Best for: workflows and automation



### Pattern 3: Fine-Tuning

Best for: consistent style, domain language, classification formats

Not always needed—often RAG + good prompting is enough.

## 6) When Should You Choose an LLM vs Another Model?

Use an **LLM** if your core job is:

- summarisation, Q&A over documents, chat support, extracting structured info from text, writing, reasoning over instructions

Use a **non-LLM deep learning model** if your core job is:

- vision detection, speech recognition, segmentation, audio classification, signal processing, time-series forecasting

Use **classic ML** if:

- structured/tabular data with clear features and limited data (often faster + easier to explain)

## 7) Libraries You Will Commonly Use

- Deep learning frameworks: **TensorFlow** (often with **Keras**) or **PyTorch**
- LLM ecosystem: **Hugging Face Transformers**
- For building apps: FastAPI + Docker (common production combo)
- If using hosted LLMs: providers like **OpenAI** (plus others) for APIs



## 8) Project Ideas

1. **Support Chatbot with RAG**
  - Ingest PDFs/FAQs, retrieve top passages, answer with citations.
  - Add “I don’t know” behaviour when docs don’t contain the answer.
2. **Resume Screener (LLM + rules)**
  - Extract skills + years from resumes into JSON.
  - Add validation rules to catch bad output.
3. **Ticket Triage + Auto-Response Drafts**
  - Classify ticket category + urgency; draft reply; route to team.
4. **Meeting Notes → Action Items**
  - Summarise transcript; extract tasks with owner + due date fields.
5. **Invoice/Receipt Extractor**
  - OCR → LLM turns messy text into structured fields (vendor, total, date).
  - Add checks: Totals must be numeric, date format validation.
6. **Vision + LLM Combo**
  - A small image model detects something (e.g., defect present),
  - LLM explains the result and writes a report.